

“SoulMi: A Web-Based Mental Wellness Platform with Facial Emotion Recognition and AI-Powered Conversational Support”

A Project Report Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF AI & DATA SCIENCE

By

P VENKATA HARSHA VARDHAN :2520089071

SALMAN AHMED :2520089070

SAI SAKETH :2520090124

Team No. 11

Under the Esteemed Guidance of

[Dr Mantri Gayatri]

Database System Engineering and Backend Development(DBSE)



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF AI&DATA SCIENCE



Declaration

The Project Report entitled **“SoulMi: A Web-Based Mental Wellness Platform with Facial Emotion Recognition and AI-Powered Conversational Support”** is a record of bona fide work of **P VENKATA HARSHA VARDHAN (2520089071)**, **SALMAN AHMED (2520089070)**, and **SAI SAKETH (2520090124)**, Team No. 11, submitted in partial fulfilment for the award of B.Tech to K L (Deemed to be) University. The results embodied in this report have not been copied from any other department/university/institute.

P VENKATA HARSHA VARDHAN – 2520089071

SALMAN AHMED – 2520089070

SAI SAKETH – 2520090124

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE (AI & DATA SCIENCE)



CERTIFICATE

This is to certify that the project-based report entitled **“SoulMi: A Web-Based Mental Wellness Platform with Facial Emotion Recognition and AI-Powered Conversational Support”** is a bona fide work done and submitted by Team No. 11 — **P VENKATA HARSHA VARDHAN (2520089071)**, **SALMAN AHMED (2520089070)**, and **SAI SAKETH (2520090124)** — in partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY**, K L (Deemed to be) University, during the academic year **2025–2026 [confirm]**.

Signature of the Guide

[Guide name and designation—to be filled in]

Signature of the Course Coordinator

[to be filled in]

Signature of the HOD

[to be filled in]

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, **[Dr Mantri Gayatri]**, for their continuous support, technical guidance, and encouragement throughout the development of this project. We also thank the Department of Computer Science (AI & Data Science) at K L (Deemed to be) University for providing the facilities and environment that made this project possible. Finally, we thank our families and classmates for their encouragement during this work.

P VENKATA HARSHA VARDHAN – 2520089071

SALMAN AHMED – 2520089070

SAI SAKETH – 2520090124

Abstract

Mental health support remains difficult for many students and young professionals to access — consultation wait times are long, stigma discourages reaching out, and most people have no easy way to notice their own mood patterns over time. **SoulMi** addresses this gap as a web- based mental wellness platform that combines manual mood check-ins, automatic facial- expression-based mood detection, a therapist directory, and an AI conversational companion into one product.

The system is built on a Flask (Python) REST API backed by a MySQL relational database, serving a static HTML/CSS/JavaScript frontend. Users register and log in through session- authenticated endpoints; their mood can be logged either manually through a mood picker, or automatically through a webcam-based facial-expression-recognition pipeline built on a mini- XCEPTION convolutional neural network (trained on the FER2013 dataset), served through a dedicated `/api/detect-mood` endpoint. A conversational AI companion, proxied server-side through the OpenRouter API so that no key is ever exposed to the browser, can optionally reference a user's real logged mood history to make its responses more grounded. The system also exposes interactive Swagger/OpenAPI documentation (via Flasgger) for every endpoint. This report documents the requirements, technology choices, system architecture, database design, implementation, testing, deployment strategy, and the real engineering challenges encountered and resolved while building SoulMi — including normalization bugs in the ML inference pipeline, a breaking API change in OpenCV 5.0, and a breaking API change in Keras 3 — along with the skills gained and the scope for future enhancement.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	1-2
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	3
3	Technology Stack	3.1 Front-end 3.2 Back-end 3.3 Database 3.4 Version Control 3.5 APIs and External Services	4-5
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer 4.3 Deployment architecture	6-7
5	Design	5.2 ER Diagram / Database schema	8-9
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic 6.3 API integration 6.4 Authentication & Authorization	10-11
7	Features	7.1 List of main features 7.2 How each feature works	12-13
8	Testing	8.1 API testing 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	14-15
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	16-17
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations / known issues	18-19
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	20
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development Tools, libraries, datasets and documentation used	21-22
13	References	Screenshots of the app	23
14	Appendices	Sample code snippets Installation/setup instructions User manual or guide Geo Tag photos with guide Review forms with guide signatures	24-29

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Access to mental health support is a well-documented gap, particularly among students and young professionals. Long waits for professional consultation, social stigma around seeking help, and a general lack of easy, judgment-free tools for everyday emotional self-awareness all contribute to people's wellbeing concerns going unnoticed until they become serious. At the same time, two technologies have matured to the point where they can meaningfully help: real-time facial-expression recognition (lightweight CNNs that can run on a laptop webcam) and large language models (now accessible through pay-per-use APIs rather than requiring a team to train one in-house).

SoulMi was conceived as a single, integrated web platform that puts these pieces together: a place where a person can check in with their mood — either by picking how they feel, or by simply looking at the camera for a few seconds — track that history over time, and talk to a supportive AI companion that is aware of their actual logged patterns rather than a generic chatbot with no memory of them.

1.2 Problem Statement

Most existing wellness or journaling apps require the user to actively and accurately self-report their mood, which many people find effortful or are reluctant to do consistently. Separately, generic AI chatbots have no persistent, structured understanding of a specific user's emotional history, and standalone facial-emotion-recognition demos rarely connect their output to anything actionable (a dashboard, a trend, a conversation). There is a gap between (a) emotion-sensing technology, (b) structured data storage of that sensed emotion over time, and (c) a conversational interface that can actually use that data — SoulMi's problem statement is to close that gap in one coherent product.

1.3 Scope of the Project

The current scope of SoulMi covers:

- User registration and session-based login
- Manual mood logging through a mood-picker UI
- Automatic mood detection from a live webcam frame via a facial-expression-recognition model
- Persisted mood history in a relational (MySQL) database
- A server-proxied AI chatbot that can optionally reference the logged mood history
- A static informational/discovery area (Home, About, Therapists directory, Resources)

- Interactive, auto-generated API documentation (Swagger/OpenAPI via Flasgger)

Out of scope for the current version: real-time video-call therapy sessions, payment/booking integration with therapists, native mobile apps, and clinical-grade diagnosis (SoulMi is explicitly a self-awareness and support tool, not a diagnostic or treatment system).

1.4 Objective(s)

- Build a working full-stack web application with a persistent relational database, replacing all client-side mock/local-storage data with a real backend API.
 - Integrate a pretrained computer-vision model for facial-expression recognition into a live product feature, with a documented path to fine-tune it on custom data.
 - Integrate a third-party LLM API securely (server-side only) to power a context-aware conversational feature.
 - Apply sound software-engineering practice: environment-based configuration, password hashing, session-based auth, CORS handling, and interactive API documentation.
- Document and reflect on the real engineering challenges encountered, as a learning record of the development process.

1.5 Importance of the Application

SoulMi lowers the effort required for a person to notice their own emotional patterns — a webcam scan takes seconds, compared to the friction of a full journal entry — while still producing structured data a user (or eventually a counsellor) could review. By keeping the AI chatbot's system prompt deliberately non-diagnostic and crisis-aware, the project also demonstrates a responsible approach to building wellness-adjacent software with generative AI.

1.6 Target Users / Audience

- University students managing academic stress and early independence
- Young professionals who want a lightweight, private way to track mood over time
- Anyone looking for a first, low-friction step before deciding whether to seek professional therapy (supported by the integrated Therapists directory)

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Component	Minimum (development)	Recommended
Processor	Dual-core, 2 GHz	Quad-core, 2.5 GHz+
RAM	8 GB	16 GB (TensorFlow model loading is RAM-sensitive)
Storage	5 GB free	10 GB free (Python venv + TensorFlow + OpenCV + dataset)
Webcam	Any USB/integrated webcam	720p or better, for facial-expression detection
Network	Broadband internet	Broadband internet (for OpenRouter AI chat calls)

2.2 Software Requirements

Layer	software
Operating System	Windows(10/11),MacOS,ios
Backend runtime	Python 3.12 (TensorFlow does not yet support 3.13+/3.14 at time of writing)
Database	MySQL Server 8.x
Browser	Google Chrome or any modern Chromium/Firefox browser (for getUserMedia webcam access)
Code editor	Visual Studio Code

2.3 Development Tools and Frameworks

Tool	Purpose
venv (Python virtual environment)	Isolates backend dependencies per project
pip	Python package installation
Flask	Backend web framework / REST API
Flasgger	Auto-generated Swagger/OpenAPI docs (/apidocs)
MySQL Workbench	Database inspection and schema management
Git & GitHub	Version control and source hosting
Python's built-in http.server	Lightweight static file server for the frontend during development
Postman / Swagger UI	Manual API endpoint testing

CHAPTER -3 TECHNOLOGY STACK

3.1 Front-end

Plain HTML5, CSS3, and vanilla JavaScript (no framework). The site pre-existed as a set of static pages (Home, About, Therapists, Resources, Register/Login, Dashboard), so a framework-free approach kept the stack simple, required no build step, and let every page be served directly by a plain static file server during development.

3.2 Back-end

Flask (Python). Chosen for minimal boilerplate, a synchronous request model that is straightforward to reason about and explain, and a mature, small ecosystem (flask-cors, cookie- based session, flasgger) that covered every requirement without extra scaffolding. Flask also integrates cleanly with TensorFlow/OpenCV running in-process for the facial-expression model, since both are just Python libraries imported and called from a route handler.

3.3 Database

MySQL (via mysql-connector-python, connection-pooled). A relational schema fits this domain well: users and their mood entries are naturally tabular, related by a foreign key, and benefit from the integrity guarantees (uniqueness on email, NOT NULL constraints) a relational engine enforces at the schema level — valuable for a course project whose explicit learning outcomes include relational schema design.

3.4 Version Control

Git, hosted on **GitHub**. The dataset (FER2013 images), the Python virtual environment, `__pycache__`, and `.env` secrets are excluded via `.gitignore` — the dataset because it is large and re-downloadable (from Kaggle), the `venv/``__pycache__` because they are machine- specific build artifacts, and `.env` because it holds credentials (database password, Flask session secret, OpenRouter API key) that must never be committed.

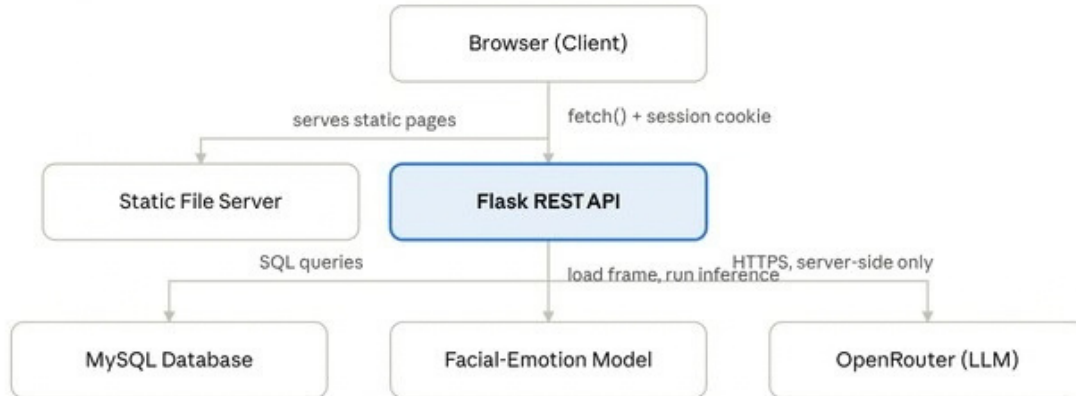
3.5 APIs and External Services

Service	Role
OpenRouter	LLM-aggregation API powering the AI chat companion; called server-side only , proxied through /api/chat, so the API key never reaches the browser
oarriaga/face_classification (public, MIT-licensed)	Source of the pretrained FER2013 mini-XCEPTION weights used for facial-expression recognition
OpenCV Haar Cascade	Face localization within a webcam frame, prior to emotion classification
Flasgger (Swagger UI)	Self-hosted interactive API documentation, no external service required

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

Browser talks only to the Flask API, which alone reaches MySQL, the emotion model, and OpenRouter



SoulMisystemarchitecture – client,API,andits threebackingservices

The browser never talks to MySQL, the emotion model, or OpenRouter directly — every one of those calls is mediated by the Flask REST API, which is the single point of authentication and authorization enforcement.

4.2 Description of Each Layer

Presentation layer (Browser / Frontend). Static HTML/CSS/JavaScript pages served by a lightweight file server during development. Responsible only for rendering UI and making `fetch()` calls to the backend with credentials: 'include' so the session cookie travels with every request.

Application layer (Flask REST API). Owns all business logic: request validation, password hashing (werkzeug.security), session management, CORS enforcement (restricted to the known frontend origin), and orchestration of the two backing capabilities below. Every route is also exposed through Flasgger's interactive Swagger documentation.

Data layer (MySQL). A pooled connection (mysql.connector.pooling.MySQLConnectionPool) holds users and mood_entries in a normalized, foreign-key-related schema (Chapter 5).

ML inference layer (in-process). The mini-XCEPTION facial-emotion model and OpenCV's Haar-cascade face detector run inside the same Flask process (not a separate microservice) — the model is loaded lazily on first use and reused for subsequent requests, which keeps deployment simple at the cost of coupling the API's memory footprint to TensorFlow's.

External AI layer (OpenRouter). A third-party LLM endpoint called only from the Flask process, never from the browser, so the API key is never exposed client-side.

4.3 Deployment Architecture

For local development, the frontend is served by Python's built-in http.server on port 5500 and the backend by Flask's development server on port 5000, run as two separate processes. For a public deployment, the intended architecture is:

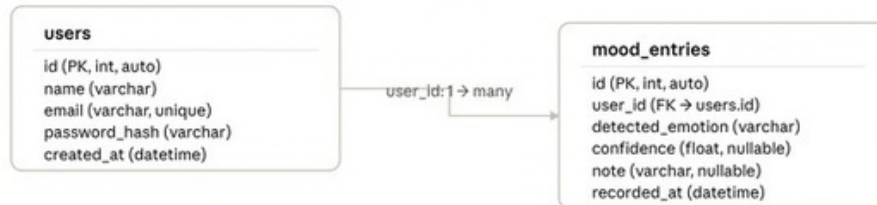
- **Frontend:** static hosting on **Netlify**, deployed directly from the GitHub repository. **Backend:**
 - **Render** (or an equivalent PaaS), running Flask behind a production WSGI server (gunicorn) rather than Flask's development server.
 - **Database:** a managed MySQL instance (Render does not offer managed MySQL natively, so an external provider such as Aiven, or a migration to Render's own PostgreSQL, is required).
- Cross-origin cookies:** since the frontend and backend then sit on different domains (not just different ports), session cookies must be reconfigured with SameSite=None; Secure, and FRONTEND_ORIGIN in the backend's environment updated to the real Netlify URL.

This two-service split (static frontend + API backend) rather than a single monolith server was a deliberate choice: it lets the frontend be deployed and cached independently (and for free, on Netlify's static tier), while the backend — which needs a real, always-on Python process for TensorFlow — is the only part that needs a paid/always-on compute tier.

CHAPTER -5 DESIGN

5.2 ER Diagram / Database Schema

users and mood_entries relate one-to-many through user_id



ER diagram – users to mood_entries, one-to-many on user_id

The schema is deliberately small and normalized: two tables related by a single foreign key, which is sufficient for the current feature set (auth + mood history) while leaving clean room to add tables (e.g. journal_entries, chat_sessions) without restructuring what already exists.

users

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR	NOT NULL
email	VARCHAR	NOT NULL, UNIQUE
password_hash	VARCHAR	NOT NULL (hashed via <code>werkzeug.security.generate_password_hash</code> , never plaintext)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

mood_entries

Column	Type	Constraints
id	INT	PRIMARY KEY, AUTO_INCREMENT
user_id	INT	FOREIGN KEY → users.id
detected_emotion	VARCHAR	NOT NULL
confidence	FLOAT	NULLABLE (set only for webcam-detected entries, not manual check-ins)
note	VARCHAR	NULLABLE (e.g. "manual check-in" or "auto-detected via webcam")
recorded_at	DATETIME	DEFAULT CURRENT_TIMESTAMP

Every mood entry — whether it came from the manual mood picker or the webcam-based /api/detect-mood endpoint — is written into this same table, distinguished only by its note field, which keeps all downstream queries (mood history, trend analysis, the AI chat's mood-context injection) simple and source-agnostic.

CHAPTER -6 IMPLEMENTATION

6.1 Module-Wise Implementation

Module	Backendfile(s)	Frontend file(s)
Authentication	app.py (/api/signup, /api/login, /api/logout db),py	register.html script.js
Mood logging (manual)	app.py (/api/mood)	dashboard.html (mood-picker widget)
Mood logging (facial)	app.py (/api/detect-mood), emotion_model.py	dashboard.html (webcam capture widget)
Mood history	app.py (/api/mood-history)	dashboard.html
AI chatbot	app.py (/api/chat)	script.js dashboard.html (chat widget, voice chat, assessment insight)
API documentation	app.py (Flasgger YAML docstrings)	— (served at /apidocs)

6.2 Front-End Logic

The frontend has no framework and no build step, so state is held in plain JavaScript variables and localStorage (used only for display convenience — e.g. caching the logged-in user's name — never as the source of truth for authentication, which is enforced server-side via the session cookie). UI rendering is done through direct DOM manipulation (document.getElementById, querySelectorAll, template literals for dynamic content) rather than a reactive framework, which was an acceptable trade-off given the site's relatively simple, mostly form-and-dashboard-driven UI.

6.3 API Integration

Every data-bearing frontend action is wired to a real backend endpoint via fetch(), always with credentials: 'include' so the Flask session cookie is sent cross-port (frontend on :5500, backend on :5000) with every request. Flask's CORS configuration (flask-cors) is restricted to the known frontend origin via the FRONTEND_ORIGIN environment variable, rather than wildcard-allowing all origins, since credentialed requests require an explicit origin.

6.4 Authentication & Authorization

- **Passwords** are never stored in plaintext — werkzeug.security.generate_password_hash / check_password_hash handle hashing and verification.

- **Sessions** are Flask's built-in signed-cookie sessions, keyed by a server-side `FLASK_SECRET_KEY` (a long random value generated with Python's `secrets.token_hex`, kept only in `.env`, never committed to version control).
- **Authorization** is enforced per-route: endpoints that touch personal data (`/api/mood`, `/api/mood-history`) check for a valid session and reject with 401 otherwise; `/api/detect-mood` works for anonymous users too (it just skips the auto-logging step if no one is logged in), since detection itself doesn't require an account.

CHAPTER -7 FEATURES

7.1 List of Main Features

- Account registration and login
- Manual mood check-in
- Webcam-based automatic mood detection
- Persistent mood history
- AI wellness chatbot
- Therapist directory
- Interactive API documentation

7.2 How Each Feature Works

1. Account registration and login.

User Perspective: a simple form (name, email, password).

Technical Description: POST /api/signup hashes the password and inserts a row into users; POST /api/login verifies the hash and starts a signed session.

2. Manual mood check-in.

User Perspective: tap one of five mood emojis on the dashboard.

Technical Description: the click handler POSTs {emotion, confidence: null, note: "manual check-in"} to /api/mood, which inserts into mood_entries under the logged-in user's id.

3. Webcam-based automatic mood detection.

User Perspective: click "Scan my mood with camera", allow camera access, click Capture.

Technical Description: the browser captures a <video> frame to <canvas>, exports it as a base64 JPEG, and POSTs it to /api/detect-mood. The server decodes the image, locates the largest face with an OpenCV Haar cascade, crops and resizes it to 64×64 grayscale, normalizes pixels to the [-1, 1] range the model was trained on, and runs it through the mini-XCEPTION model to get a softmax distribution over 7 emotions. The top prediction and its confidence are returned and, if the caller is logged in, auto-logged to mood_entries.

4. Persistent mood history.

User Perspective: a mood-trend view on the dashboard.

Technical Description: GET /api/mood-history returns the last 100 mood_entries rows for the logged-in user, ordered by recorded_at.

5. AI wellness chatbot.

User Perspective: a chat bubble available site-wide.

Technical Description: every message is sent to POST /api/chat, which (optionally, with the user's consent toggle) enriches a fixed, crisis-aware system prompt with the user's last 7 real mood entries pulled from MySQL, then forwards the conversation to OpenRouter and returns the model's reply — the OpenRouter API key itself is read only from server-side environment variables and is never sent to or visible from the browser.

6. Therapist directory.

User Perspective: a filterable list of therapist profiles (location, specialization, language). **Technical Description:** currently static content on therapists.html.

7. Interactive API documentation.

User Perspective / Technical Description: /apidocs (Flasgger/Swagger UI) lists every backend endpoint with its expected request body and response shape, and lets a developer send real test requests from the browser.

CHAPTER -8 TESTING

8.1 API Testing

Every endpoint was manually tested through Flasgger's Swagger UI (/apidocs) and via direct browser/DevTools inspection of request and response bodies, since Swagger UI's "Try it out" exercises the real running server rather than a mocked one. Each endpoint's expected status codes (200/201 success, 400 validation error, 401 unauthenticated, 503 dependency not ready) were verified against actual responses during development.

8.2 Integration Testing

Full user-journey testing was performed end-to-end through the real UI: register → login → manual mood log → webcam mood scan → mood history → AI chat (with and without mood- context sharing enabled) → logout, confirmed against the actual MySQL rows created at each step (verified directly via SELECT queries in MySQL Workbench) rather than trusting the UI alone.

8.3 Tools Used for Testing

Tool	Used for
Flasgger / Swagger UI Chrome	Interactive endpoint testing
DevTools (Network, Console)	Inspecting request/response payloads, diagnosing CORS and session-cookie issues
MySQL Workbench	Verifying database state directly against API behaviour
A custom evaluate_emotion_model.py script	Computing accuracy, per-class precision/recall/F1, and a confusion matrix for the facial-emotion model against a held-out labelled test set (FER2013)

8.4 Test Cases and Results

#	Testcase	Expected result	Actual result
1	Signup with a new email	201, row created in users	Pass
2	Signup with an already-registered email	409 conflict	Pass
3	Login with correct credentials	200, session cookie set	Pass
4	Login with wrong password	401	Pass
5	POST /api/mood while logged out	401	Pass
6	POST /api/mood while logged in	201, row in mood_entries	Pass

#	Test case	Expected result	Actual result
7	POST/api/detect-mood with a clear frontal face	200, plausible emotion + confidence	Pass (after fixing the [-1,1] normalization bug — see Chapter 10)
8	POST /api/detect-mood with no face in frame	200, faces_found: 0	Pass
9	POST /api/chat with a valid OpenRouter key/model	200, non-empty reply	Pass
10	POST /api/chat with include_mood_context: true	Reply can reference real logged mood history	Pass
11	Cross-origin request from an unauthorized origin	Blocked by CORS	Pass

Model evaluation on a held-out FER2013 test split (via `evaluate_emotion_model.py`) is documented in Appendix 14 along with the resulting accuracy, per-class F1 scores, and confusion matrix for the specific weights deployed.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy (Local Development)

- 1.cd backend && python -m venv venv && venv\Scripts\activate (Windows)
– Python 3.12, since TensorFlow does not yet support 3.13+/3.14.
2. pip install -r requirements.txt (installs Flask, TensorFlow, OpenCV, Flasgger, etc.)
- 3.Run schema.sql against a local MySQL instance to create the database and tables.
4. Copy .env.example to .env and fill in MYSQL_PASSWORD, a generated FLASK_SECRET_KEY, and an OPENROUTER_API_KEY.
- 5.python download_pretrained_model.py to fetch the base FER2013 mini-XCEPTION weights.
- 6.python app.py (backend, port 5000) and, in a second terminal from the project root, python -m http.server 5500 (frontend).

9.2 Environment Configuration

All environment-specific values live in backend/.env (excluded from Git via .gitignore),

templated by .env.example:

Variable	Purpose
MYSQL_HOST / PORT / USER / PASSWORD / DATABASE	Database connection
FLASK_SECRET_KEY	Signs session cookies
FRONTEND_ORIGIN	CORS allow-list
OPENROUTER_API_KEY / OPENROUTER_MODEL	AI chat provider credentials and model selection

9.3 Hosting of Frontend and Backend (Planned Production Deployment)

Layer	Planned host	Notes
Frontend	Netlify	Static hosting, deployed from the GitHub repo
Backend	Render	Needs a production WSGI server (gunicorn) in place of Flask's dev server; the free tier's 512 MB RAM limit is tight for TensorFlow and may require a paid tier for reliable live demos
Database	External managed MySQL (e.g. Aiven's free tier)	Render offers managed PostgreSQL but not MySQL, so either an external MySQL host is used, or the schema is ported to PostgreSQL

Moving from local to cross-domain hosting also requires: setting session cookies to SameSite=None; Secure (since Netlify and Render sit on different domains, not just different localhost ports), updating FRONTEND_ORIGIN to the real Netlify URL, and updating the frontend's API_BASE constant to the deployed Render URL.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Keras 3's SeparableConv2D API change.** The installed TensorFlow version (2.21) ships Keras 3, which renamed SeparableConv2D's `kernel_regularizer` argument to `separate_depthwise_regularizer / pointwise_regularizer` arguments. The model code, written against the older Keras 2 API, failed to build at all (Unrecognized keyword arguments).

- **OpenCV 5.0 removed the classic Haar-cascade API.** `pip install opencv-python-headless` pulled the newest major version (5.0), which moved `cv2.CascadeClassifier` out of the base package into a separate contrib module, causing `AttributeError: module 'cv2' has no attribute 'CascadeClassifier'` at runtime.

A silent normalization mismatch collapsed every facial prediction to "neutral".

- The inference code scaled pixels to `[0, 1]`, but the pretrained weights (from the oarriaga/face_classification project) were trained on pixels scaled to `[-1, 1]`. The model loaded and ran without error, so the bug was invisible until predictions on real faces were checked and found to be degenerate — every input, regardless of actual expression, biased toward the same class.

A deprecated OpenRouter free-model slug. `meta-llama/llama-4-`

- `maverick:free` stopped being served (404/402 errors from the provider), requiring a switch to OpenRouter's own auto-routing `openrouter/free` model id to avoid depending on any single free-tier slug's continued availability.

Cross-origin session cookies. Early testing intermittently "logged users out" because

- requests were made with the frontend opened via `file://` instead of through a proper HTTP origin, which silently fails CORS's credentialed-request requirements.

A misleading UI state, not an actual backend bug. Several static pages (Home,

- Therapists, etc.) had a hardcoded "Get Started" button with no awareness of whether the visitor was already logged in, making it look like navigating away logged the user out, when in fact the session was untouched.

10.2 Solutions Applied

- Updated `emotion_model.py` to pass `depthwise_regularizer` and `pointwise_regularizer` instead of `kernel_regularizer` on every `SeparableConv2D` call.
- Pinned `opencv-python-headless<5` in `requirements.txt` to stay on the last 4.x release line, which still bundles the classic Haar-cascade detector.

- Added the missing $(\text{pixel}/255 - 0.5) * 2.0$ normalization step to both the inference pipeline (`emotion_model.py`) and the fine-tuning script (`train_emotion_model.py`), so both stay consistent with how the base weights were trained.
- Switched `OPENROUTER_MODEL` to `openrouter/free`.
- Standardized on serving the frontend through `python -m http.server 5500` (a real HTTP origin) rather than opening files directly, and ensured every `fetch()` call passes credentials: `'include'`.
- Added a small script-level check (in `script.js`) that swaps any "Get Started" button to a "Dashboard" button whenever a logged-in user is detected, across every public page.

10.3 Current Limitations / Known Issues

- The Haar-cascade face detector is fast and dependency-light but less robust than modern DNN-based detectors (e.g. MTCNN, RetinaFace) under poor lighting, side angles, or partial occlusion.
- The base emotion model is trained on FER2013 (lab-condition, mostly frontal photos); real webcam accuracy under varied lighting is noticeably lower until the model is fine-tuned on the users' own conditions, which the project supports (`train_emotion_model.py`) but has not yet been exercised against a large custom dataset.
- Flask's built-in development server (`app.run(debug=True)`) is not production-grade — a real deployment needs a WSGI server such as gunicorn behind it. `openrouter/free` routes to whichever free-tier model is currently available, so response quality and availability can vary over time; this is mitigated, not eliminated. There is no automated test suite yet — testing has been manual (Swagger UI, DevTools, direct SQL verification) rather than CI-integrated `pytest/unittest` coverage.
- The therapist directory is static content, not yet backed by its own database table or booking flow.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

- An encrypted private journal (passphrase-protected notes separate from the mood/chat data).
- A proper sleep_entries or broader wellness-log table, so the AI chat's mood-context injection can reason about more than mood alone.
- A booking/scheduling flow for the Therapist directory, backed by its own database table instead of static content.
- A fine-tuned, user- or cohort-specific emotion model, trained on consenting users' own webcam captures to close the accuracy gap between FER2013's lab conditions and real-world usage.
- An automated test suite (pytest) and a CI pipeline (GitHub Actions) running it on every push.

11.2 Possible Integrations or Optimizations

- Replace the Haar-cascade face detector with a modern DNN-based one (e.g. MTCNN) for better accuracy under poor lighting and non-frontal angles.
- Move the AI chat provider behind an abstraction so alternative providers/models can be swapped in without code changes beyond .env.
- Add response caching or a lighter always-warm inference path to reduce the facial- detection endpoint's cold-start latency (first request after server start is slow while TensorFlow loads the model into memory).
- Containerize the backend (Docker) to make the TensorFlow/OpenCV dependency set reproducible across machines, instead of relying on a manually created virtual environment.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

SoulMi set out to close the gap between emotion-sensing technology, structured data storage, and a conversational interface that can actually use that data — delivered as a single, coherent web platform rather than three disconnected demos. The result is a working full-stack application: a Flask REST API over a normalized MySQL schema, a facial-expression- recognition pipeline built on a fine-tunable mini-XCEPTION model, and a server-proxied, mood-aware AI chat companion, all documented through interactive Swagger API docs.

12.2 What Was Achieved

- A fully functional authentication system with hashed passwords and session-based authorization.
- A real, persisted mood-tracking feature with two independent input paths (manual and facial-recognition-based) writing to one unified schema.
- A working end-to-end computer-vision pipeline — from webcam frame, through face detection and a CNN, to a stored, queryable result.
- A secure integration pattern for a third-party LLM API that keeps all secrets server-side. A documented, reproducible local development setup and a concrete plan for public deployment (Netlify + Render).
- A genuine debugging record (Chapter 10) covering three separate breaking third-party API changes (Keras 3, OpenCV 5.0, an OpenRouter model deprecation) encountered and resolved during the project.

12.3 Skills Learned During Development

- Designing a normalized relational schema and integrating it into a live backend, replacing frontend mock/local-storage data entirely.
- Building and documenting a REST API (Flask, Flasgger/OpenAPI), including authentication, CORS, and environment-based configuration.
- Working with a pretrained computer-vision model end-to-end: loading third-party weights into a reconstructed architecture, diagnosing a silent preprocessing bug through direct experimentation rather than guesswork, and building an evaluation script to measure accuracy and F1 objectively.
- Integrating a third-party LLM API securely, understanding why client-side API keys are a real security risk and how to avoid it.

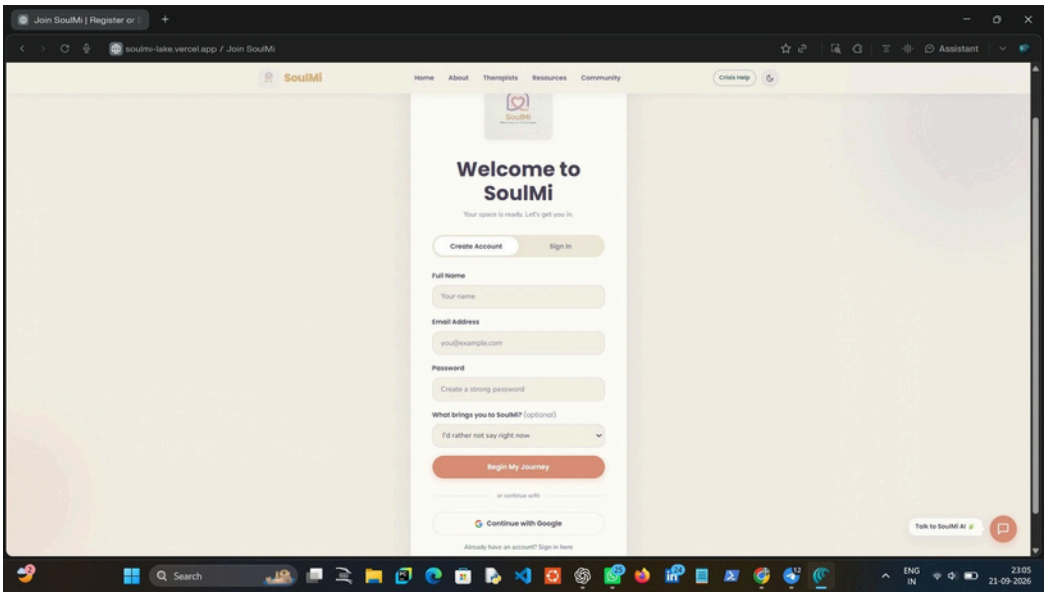
- Practical debugging across a full stack — Python/Flask tracebacks, browser DevTools, MySQL, and dependency-version conflicts between fast-moving libraries (TensorFlow/Keras, OpenCV).
Deployment planning for a two-service (static frontend + API backend) architecture, including the
- cross-origin cookie and environment-configuration changes that distinguish local development from a public deployment.

CHAPTER -13 REFERENCES

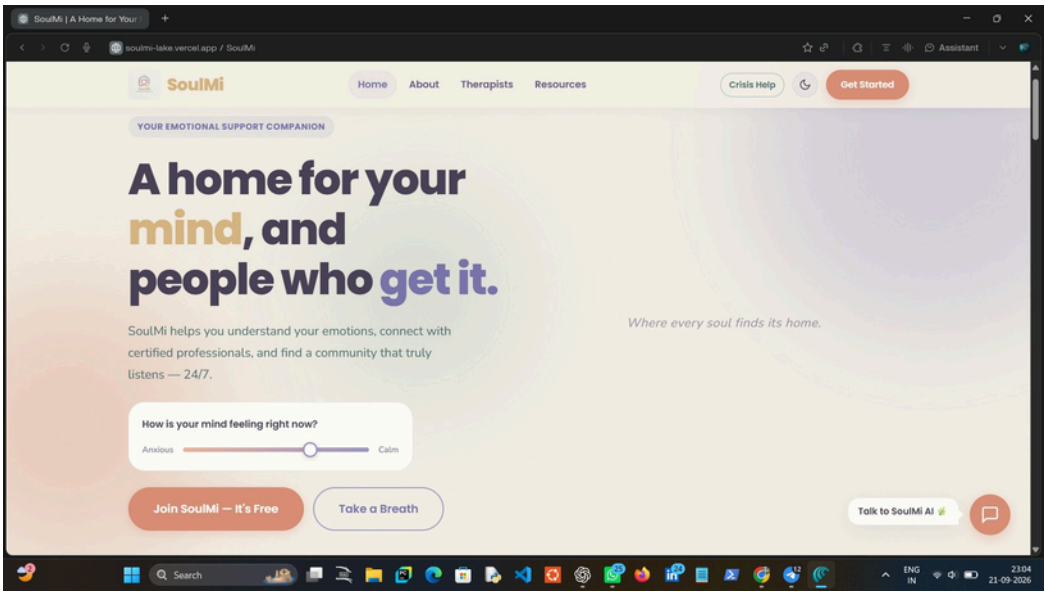
- Flask documentation — flask.palletsprojects.com
- Flasgger documentation — github.com/flasgger/flasgger
- Flask-CORS documentation — flask-cors.readthedocs.io
- MySQL Connector/Python documentation — dev.mysql.com/doc/connector-python
- TensorFlow / Keras 3 API documentation — keras.io and tensorflow.org
- OpenCV documentation — docs.opencv.org
- Goodfellow, O. et al., **FER2013** facial-expression dataset (used for base model training and evaluation) — distributed via Kaggle: kaggle.com/datasets/msambare/fer2013
- Arriaga, O., face_classification (mini-XCEPTION architecture and pretrained FER2013 weights, MIT License) — github.com/oarriaga/face_classification
- OpenRouter API documentation — openrouter.ai/docs
- Werkzeug security utilities documentation — werkzeug.palletsprojects.com
- MDN Web Docs — MediaDevices.getUserMedia(), Canvas API, Fetch API — developer.mozilla.org

CHAPTER -14 APPENDICES

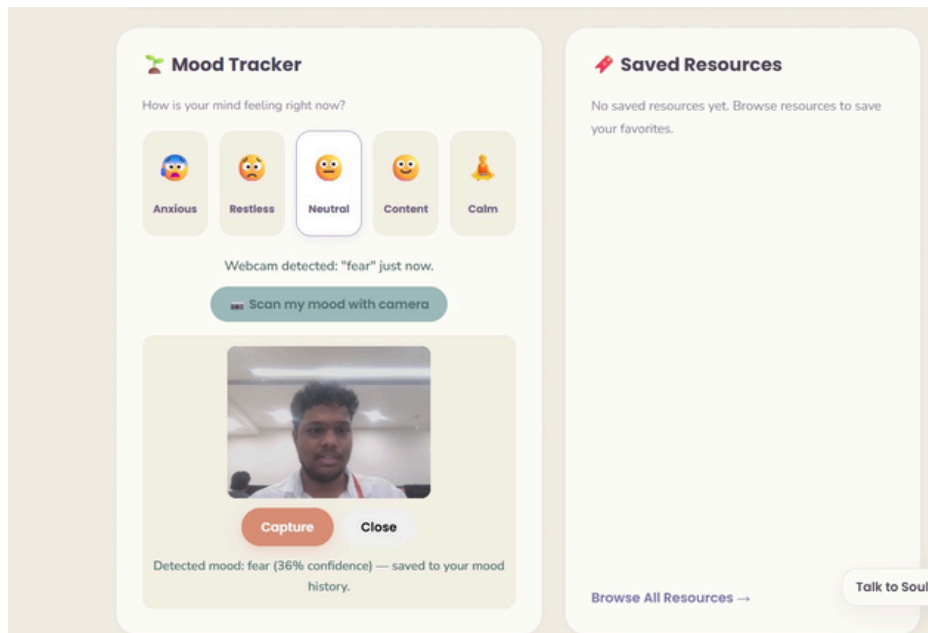
Screenshots of the App



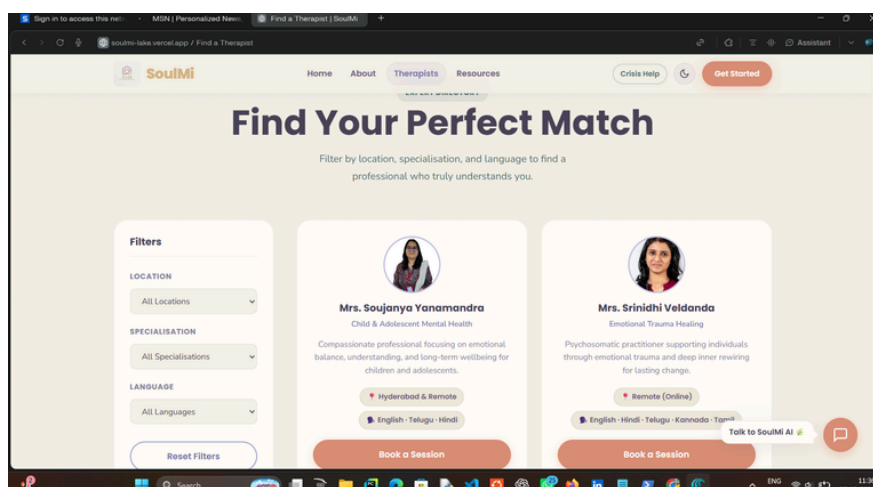
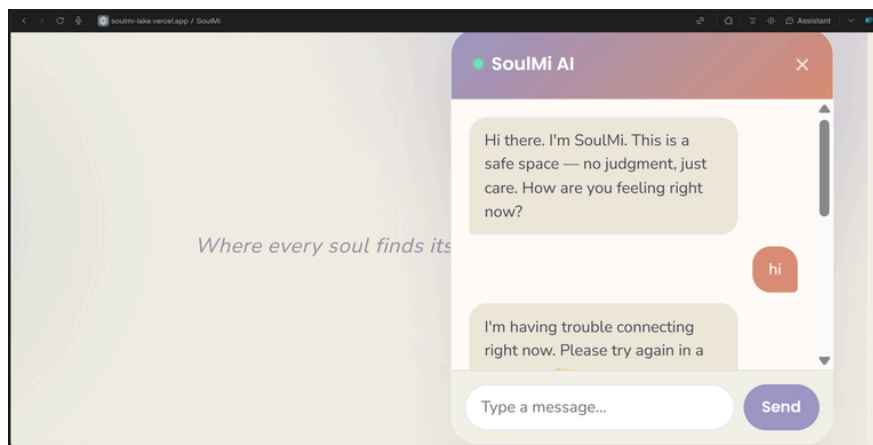
Registration page ("Welcome to SoulMi") – Create Account / Sign In



Landing page – "A home for your mind, and people who get it."



Dashboard Mood Tracker card with the "Scan my mood with camera" webcam feature



Sample Code Snippets

Facial-expression detection endpoint (backend/app.py):

```
@app.route("/api/detect-mood", methods=["POST"])
def detect_mood():
    data = request.get_json(force=True)
    image_data_url = data.get("image", "")
    if "," in image_data_url:
        image_data_url = image_data_url.split(",", 1)[1]
    img_bytes = base64.b64decode(image_data_url)
    bgr_image = cv2.imdecode(np.frombuffer(img_bytes, np.uint8), cv2.IMREAD_COLOR)
    results = detect_faces_and_emotions(bgr_image, max_faces=1)
    if not results:
        return jsonify({"faces_found": 0}), 200
    top = results[0]
    user_id = current_user_id()
    if user_id:
        # auto-log to mood_entries
        ...
    return jsonify({"emotion": top["emotion"], "confidence": top["confidence"]})
```

Pixel normalization fix (backend/emotion_model.py):

```
face_roi = face_roi.astype("float32") / 255.0
face_roi = (face_roi - 0.5) * 2.0 # scale to [-1, 1] to match training
```

Server-side AI chat proxy (backend/app.py , abridged):

```
@app.route("/api/chat", methods=["POST"])
def chat():
    api_key = os.getenv("OPENROUTER_API_KEY", "")
    data = request.get_json(force=True)
    messages = data.get("messages") or [{"role": "user", "content": data.get("prompt", "")}]
    resp = upstream_requests.post(
        "https://openrouter.ai/api/v1/chat/completions",
        headers={"Authorization": f"Bearer {api_key}"},
        json={"model": os.getenv("OPENROUTER_MODEL"), "messages": messages},
    )
    return jsonify({"reply": resp.json()["choices"][0]["message"]["content"]})
```

Installation / Setup Instructions

See Chapter 9, Section 9.1 for the full step-by-step local setup (virtual environment, pip install, schema.sql, .env configuration, running both servers).

User Manual / Guide

- Open the site and click **Get Started** to register, or **Log In** if you already have an account.
- On the **Dashboard**, use the mood-emoji picker for a quick manual check-in, or click  **Scan my mood with camera** to let the facial-expression model detect it automatically.
- View your mood trend under **Mood & Analytics**.
- Click the chat bubble (bottom-right, any page) to talk to the **SoulMi AI** companion; optionally enable mood-sharing so it can reference your real logged history.
- Browse the **Therapists** directory to find a professional, filterable by location, specialization, and language.
- Use **Crisis Help** (top navigation, always visible) if you or someone you know needs immediate support.